Task 1 — Web App QA & Debug Report

Prepared by: Nayanaviram Panigrahi
App tested: RealWorld “Conduit” API, `gothinkster/node-express-realworld-example-app` (Express + Prisma + PostgreSQL 15), run locally on port 3000.
Why not the live demo: `demo.realworld.io` returned 404 and its API returned Cloudflare error 1016 on the day of testing.
Scope and limits: this repo is backend only, so the flows were exercised through the HTTP API with curl and by reading the source: sign-up, login, profile update, article create and edit, follow, comment validation, and edit/delete attempts by a non-owner. A successful owner delete and a successful comment post were not run. No browser UI, accessibility or visual checks were possible. There is no logout endpoint, because auth is a stateless JWT.
Result: 10 issues found (1 Critical, 3 High, 4 Medium, 2 Low). Each was reproduced against the running server; the suspected cause comes from reading the source.

Bug table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	**Global article feed is empty for visitors; logged-in users see only their own articles**	1. Register two users and create an article as each. 2. `GET /api/articles` with no token. 3. Repeat with user A's token.	**Expected:** every published article appears in the global feed for everyone. **Actual:** anonymous call returns `{"articles":[],"articlesCount":0}`. User A sees only A's articles. `GET /api/tags` is also empty.	**Critical**	Query in `article.service.ts` (buildFindAllQuery) restricts authors to `demo = true` OR the caller's own id. Same `demo: true` filter in `tag.service.ts`.
2	**Titles in non-Latin scripts (e.g. Hindi) all get the same slug; the second one is rejected as "must be unique"**	1. As one user, POST an article titled "■■■■ ■■■■ ■■■". 2. POST a different article titled "■■■■■ ■■■■■■ ■■■ ■■■".	**Expected:** both save with distinct URLs. **Actual:** first gets slug `-2`; second returns 422 `title: must be unique`. Symbol-only titles behave the same (`!!-2`).	**High**	Slug is `slugify(title) + '-' + userId`. slugify drops non-Latin characters, leaving an empty string. The suffix is the user id, not the article id, so it cannot disambiguate.
3	**Changing email or username to one already taken returns HTTP 500 with a raw database error and a server file path**	1. Register users A and B. 2. As A, `PUT /api/user` with B's email.	**Expected:** 422 `email: has already been taken`, as registration does. **Actual:** 500 whose body is the Prisma error text, including `C:\...\auth.service.ts:159` and the failed constraint.	**High**	`updateUser` has no uniqueness check (unlike `createUser`). The global error handler returns `err.message` for any unknown error, leaking internals.
4	**Registration accepts an invalid email, a 1-character username and a 1-character password**	1. `POST /api/users` with email "not-an-email", username "a", password "1".	**Expected:** 422 with field errors. **Actual:** 201 Created and a valid token.	**High**	`createUser` only checks the fields are non-empty. No format or length rules.
5	**Invalid pagination values give wrong results or a 500**	1. As a user with 3 articles: `GET /api/articles?limit=-1`. 2. `GET /api/articles?offset=-1`.	**Expected:** 422 or a clamped default. **Actual:** limit=-1 returns 1 article (the oldest) while `articlesCount` says 3. offset=-1 returns 500 with the Prisma invocation text.	**Medium**	`Number(query.limit) || 10` passes negatives straight to Prisma (`take` < 0 reads backwards; negative `skip` throws).
6	**Malformed or oversized JSON bodies return 500 instead of 400/413**	1. `POST /api/users` with body `{"user":`. 2. POST an article with a 200 KB body.	**Expected:** 400 for bad JSON, 413 for too large. **Actual:** both return 500 with the parser's message as the body.	**Medium**	`body-parser` errors carry a status but the error handler in `main.ts` ignores it and falls to the generic 500 branch. Default 100 KB limit is also not documented.
7	**Wrong login status code and no brute-force protection**	1. `POST /api/users/login` with a wrong password 30 times in a row.	**Expected:** 401 (or 422) and throttling or lock-out after repeated failures. **Actual:** every attempt returns 403 immediately; none were slowed or blocked.	**Medium**	403 is hard-coded in `login`. No rate limiter middleware is installed.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
8	**Editing an article title silently changes its URL; the old link returns 404**	1. Create "Hello" (slug Hello-2). 2. `PUT` a new title. 3. Open the old slug.	**Expected:** the existing link keeps working. **Actual:** slug becomes `Hello-renamed-2`; `GET /api/articles/Hello-2` returns 404.	**Medium**	`updateArticle` recomputes the slug from the title and keeps no redirect or stable id.
9	**HTML in title and body is stored and returned unescaped**	1. POST an article with title `<script>alert(1)</script>` and body `<img src=x onerror=alert(1)>`.	**Expected:** content is rejected or sanitised, or the client escapes on render. **Actual:** stored as typed (201). Safe only if every client escapes it. Not tested in a browser because this repo has no frontend.	**Low**	No server-side sanitisation. Risk depends on whichever frontend renders the data.
10	**A user can follow themselves; tokens last 60 days with no server-side revoke**	1. `POST /api/profiles/alice/follow` as alice. 2. Decode the JWT: exp minus iat is 5,184,000 s.	**Expected:** self-follow refused; a way to end a session. **Actual:** self-follow returns 200 and following: true. No logout or revoke endpoint exists.	**Low**	No self-check in the follow handler. Stateless JWT with a long expiry and no deny-list.

Root-cause analysis — Issue 1: global feed shows no articles

Anonymous visitors get an empty article list, and a logged-in user sees only articles they wrote themselves, even though the database holds articles from several users. The cause is in `buildFindAllQuery` in `article.service.ts`. It always adds a condition that the article's author must have `demo = true`, or must be the logged-in user. Users created through the normal `POST /api/users` route have `demo = false` by default, so the first condition never matches them. The tag list in `tag.service.ts` has the same `demo: true` filter, which is why `/api/tags` was also empty after I created an article with a tag. This looks like a guard meant for a public showcase site, where only seeded demo content should be shown, and it was left on for every deployment. For a real product it makes the main page look broken: new users sign up and see nothing. I would fix it by removing the author filter from the default query and keeping only the optional `author`, `tag` and `favorited` filters. If the demo behaviour is still wanted, it should sit behind an environment flag such as `DEMO_MODE=true`. I would add an integration test that creates two non-demo users and asserts both articles appear in an anonymous `GET /api/articles`.

Observations that were not bugs

Authorisation held up: another user's edit and delete both returned 403 and the article was unchanged; missing or invalid tokens returned 401; blank article and comment fields returned 422; an unknown slug returned 404. The `Bearer` scheme is accepted as well as `Token`, which is harmless.

Environment

Windows 11, Node 22.17, PostgreSQL 15 in Docker (host port 5433), Prisma 4.16 migrations applied with `prisma migrate deploy`, server started with `npx nx serve api`. Test accounts were throw-away values created for this run.